

Guide Détaillé — Ameni

Yocto Project + Build Runner Docker (Zephyr) + Agent 2 + CI/CD

Ton périmètre en un coup d'œil

Élément	Détail
OS embarqué	Image Linux minimale via Yocto Project (testée d'abord via QEMU)
CI/CD	Conteneur "Build Runner" Docker qui compile un projet Zephyr avec <code>west</code> + pipeline GitHub Actions
Agent IA	Agent 2 — Architecte (décide stratégie de build + OTA)
Autre	Tests automatisés, documentation

Rappel important : Yocto est objectivement la partie la plus difficile du projet. **Tu as un plan de secours officiel** (Raspberry Pi OS standard) — voir section risques du cahier des charges. N'hésite pas à t'appuyer dessus si le temps presse, ce n'est pas un échec.

Semaine 1-2 — Premiers pas Yocto (à distance, via QEMU)

■ **Yocto Project** : un ensemble d'outils pour construire une distribution Linux sur-mesure, adaptée à un matériel précis. ■ **BitBake** : le moteur de build de Yocto — il lit des "recettes" et assemble l'image Linux finale à partir de milliers de petits paquets. ■ **Recette (recipe)** : un fichier qui décrit comment construire un composant logiciel particulier (ex : comment compiler et installer Docker Engine). ■ **Layer (couche Yocto)** : un ensemble de recettes regroupées par thème (ex : une couche pour les pilotes graphiques, une couche pour les outils réseau). On combine plusieurs couches pour construire son image. ■ **QEMU** : un émulateur qui permet de faire tourner un système Linux complet (le résultat de ton image Yocto) directement sur ton ordinateur, sans avoir besoin du vrai Raspberry Pi.

Étape 1 — Installer l'environnement (sur une VM Linux ou WSL2 si tu es sous Windows)

```
bash
```

```
sudo apt update
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
  chrpath socat cpio python3 python3-pip python3-pexpect xz-utils debianutils \
  iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint \
  xterm rsync
```

Étape 2 — Récupérer Poky (la distribution de référence Yocto) et une couche Raspberry Pi

```
bash
git clone -b scarthgap git://git.yoctoproject.org/poky
cd poky
git clone -b scarthgap git://git.yoctoproject.org/meta-raspberrypi
```

 **Poky** : la distribution de référence fournie par le Yocto Project, qui sert de point de départ pour construire sa propre image.

Étape 3 — Configurer et lancer un premier build minimal (pour QEMU, pas encore le vrai Pi)

```
bash
source oe-init-build-env
bitbake core-image-minimal
```

 **Attention aux délais** : cette première commande peut prendre **plusieurs heures** selon ta machine (elle télécharge et compile des milliers de paquets). Lance-la en début de journée et travaille sur autre chose en parallèle (lecture de doc, prompt engineering) pendant qu'elle tourne.

Étape 4 — Tester dans QEMU

```
bash
runqemu qemux86-64
```

Livraison de la semaine 1-2 : une image Linux minimale démarre dans QEMU, même sans encore Docker dedans.

Semaine 3 — Build Runner Docker (compilation Zephyr isolée)

 **Build Runner** : un conteneur Docker dédié uniquement à la compilation — il ne fait tourner aucun service, il sert juste d'environnement isolé et reproductible pour construire le firmware.

dockerfile

```
# build-runner/Dockerfile
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y \
    git cmake ninja-build gperf ccache dfu-util device-tree-compiler \
    python3-dev python3-pip python3-setuptools wget curl xz-utils file

RUN pip3 install west

WORKDIR /workspace

RUN west init -m https://github.com/zephyrproject-rtos/zephyr --mr main .
RUN west update
RUN west zephyr-export
RUN pip3 install -r zephyr/scripts/requirements.txt

ENTRYPOINT ["west", "build"]
```

Construire et tester :

bash

```
docker build -t build-runner ./build-runner
docker run --rm -v $(pwd)/mon_projet_zephyr:/workspace/app build-runner -b esp32
```

Livrable : le conteneur compile un exemple Zephyr simple (ex : `samples/basic/blink`) et produit un binaire, ou affiche une erreur exploitable.

Semaine 4 — Agent 2 (Architecte)

Contrat d'interface reçu de Sahar (Agent 1) :

json

```
{
  "framework": "Zephyr RTOS",
  "fichiers_detectes": ["prj.conf", "CMakeLists.txt"],
  "carte_cible": "esp32",
  "protocoles": ["MQTT", "WiFi"]
}
```

Crée ce fichier toi-même en `mock_agent1_output.json` et commence sans attendre Sahar.

python

```
import json
import anthropic

client = anthropic.Anthropic(api_key="TA_CLE_API")

def decider_architecture(analyse: dict) -> dict:
    prompt = f"""
    À partir de cette analyse de projet embarqué :
    {json.dumps(analyse)}

    Décide de la stratégie technique. Réponds UNIQUEMENT en JSON :
    {{
        "strategie_build": "docker_west_build",
        "ota_active": true/false,
        "monitoring": true/false,
        "broker_mqtt": "mosquitto",
        "justification": "..."
    }}
    """

    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

with open("mock_agent1_output.json") as f:
    analyse_mock = json.load(f)

decision = decider_architecture(analyse_mock)
print(decision)
```

Livrable : publie `mock_agent2_output.json` pour Nour dès que stable.

Semaine 5 — Build Runner complet + Pipeline CI/CD

Build Runner complet

Fais compiler un vrai projet Zephyr fourni par Nour (pas juste l'exemple `blinky`), avec extraction du binaire ou du log d'erreur exploitable par l'Agent 4 de Sahar.

Pipeline CI/CD (transféré depuis Sahar, maintenant ta responsabilité)

yaml

```
# .github/workflows/ci.yml
name: CI Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests/

  build-docker:
    runs-on: ubuntu-latest
    needs: test-backend
    steps:
      - uses: actions/checkout@v4
      - run: docker compose build

  build-firmware:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Construire le Build Runner
        run: docker build -t build-runner ./build-runner
      - name: Compiler le firmware Zephyr
        run: docker run --rm -v ${GITHUB_WORKSPACE}/firmware:/workspace/app
```

Livrable : badge de statut CI dans le README, visible dans l'onglet Actions du dépôt.

Semaine 6 — Optimisation Yocto + support OTA

Intégrer Docker dans l'image Yocto

Ajoute la couche `meta-virtualization` (qui contient les recettes Docker) à ta configuration :

```
bash
```

```
git clone -b scarthgap git://git.yoctoproject.org/meta-virtualization
```

Dans `conf/bblayers.conf`, ajoute le chemin vers cette couche, puis dans `conf/local.conf`:

```
IMAGE_INSTALL:append = " docker-ce"
```

Relance le build (`bitbake core-image-minimal`) — attends-toi à un temps de compilation long, relance-le en tâche de fond.

Si le temps presse : bascule sur le plan de secours (Raspberry Pi OS standard + `apt install docker.io`) et documente honnêtement dans le rapport où tu en étais avec Yocto — c'est parfaitement valorisable même incomplet.

■ **OTA (Over-The-Air)** : mise à jour du firmware à distance, sans câble. "Support OTA basique" veut dire ici : documenter la stratégie possible plutôt que l'implémenter entièrement — c'est un axe d'amélioration future acceptable à ce stade du projet.

Livable : soit l'image Yocto avec Docker démarre dans QEMU (ou sur le vrai Pi), soit le plan de secours est en place et documenté.

Semaine 7 — Intégration finale

Avec Sahar et Nour : brancher le Build Runner au backend (Sahar/Nour), valider l'isolation des builds, tests de bout en bout avec de vrais projets contenant des fautes volontaires.

Ta soupape de sécurité

1. Sacrifie en premier : l'optimisation Yocto avancée (semaine 6) — bascule sur le plan de secours sans culpabilité.
2. Sacrifie ensuite : le support OTA (documentation suffit, pas besoin d'implémentation complète).
3. Ne sacrifie jamais : la publication de `mock_agent2_output.json` — Nour en a besoin pour ne pas être bloquée.

Combine ce document avec le cahier des charges fusionné pour la vue d'ensemble.